CODING_RULES.md

General

- No commented-out dead code committed. Delete it; git history keeps it if needed.
- Every non-trivial function gets a one-line docstring/comment stating *why*, not what (the code already says what).
- No magic strings for event types / status values — use enums (Python `Enum`, TS `type Status = "sending" | "sent" | "delivered" | "read"`).
- Keep functions under ~40 lines; extract helpers when a function is doing more than one job.

Backend (Python / FastAPI)

- Formatting: `black` + `isort`, line length 88 (black default). `ruff` for linting.
- Naming: `snake_case` for functions/variables, `PascalCase` for classes/Pydantic models/ORM models, `UPPER_SNAKE` for constants.
- File naming: `snake_case.py`, one router per domain (`routes_<domain>.py`), one service module per domain (`<domain>_service.py`).
- Every route function is `async def` and type-hinted on both params and return.
- Pydantic schema naming: `<Domain>Create`, `<Domain>Read`, `<Domain>Update` (e.g. `MessageCreate`, `MessageRead`) — never reuse the ORM model as the response model.
- DB access only inside `services/*` or `db/*` — never a raw query inside a route.
- All timestamps stored in UTC; conversion to local time happens on the frontend.
- Errors: raise `HTTPException` with a clear `detail` string; don't let raw exceptions/tracebacks reach the client.

Frontend (Next.js / TypeScript)

- Formatting: `prettier` (default config) + `eslint` (next/core-web-vitals).
- Naming: `PascalCase` for components and their files (`MessageBubble.tsx`), `camelCase` for functions/variables/hooks, hooks always prefixed `use` (`useConversationStore`).
- Components: functional only, no class components. Props typed with an explicit `interface <Component>Props`.
- No `any`. If a type is genuinely unknown, use `unknown` and narrow it.
- One component = one file. Co-locate a component's tiny helper sub-components in the same folder, not the same file, once the file exceeds ~150 lines.
- All API/WebSocket calls go through `lib/api-client.ts` / `lib/ws-client.ts` — never a raw `fetch`/`new WebSocket()` inside a component.
- Styling: Tailwind utility classes; no inline `style={}` except for values that are computed at runtime (e.g. dynamic avatar colors).
- Client vs Server components: default to Server Components; add `"use client"` only where interactivity/state/WebSocket is actually needed.

Git / commits

- Conventional commits: `feat:`, `fix:`, `refactor:`, `docs:`, `chore:`, `test:`.
- One logical change per commit; no "wip" or "final final" commits in the submitted history.
- Branch naming (if branches are used): `feat/<short-name>`, `fix/<short-name>`.

Tests (lightweight, given the 24h budget)

- Backend: at minimum, service-layer unit tests for message send/read-status transitions and group add/remove-member permission checks.
- Frontend: skip exhaustive testing given the time budget; note this explicitly as an assumption in the README rather than silently skipping it.

Documentation discipline

- Any deviation from PROJECT_CONTEXT.md or ARCHITECTURE.md gets logged in FEATURE_LOG.md's Decisions section the same day it's made, with a one-line reason.